Implementando Integração Contínua (CI) com GitHub Actions

Objetivo: Configurar um pipeline de Integração Contínua em uma API Python (FastAPI), forçar uma falha para entender o papel do CI na segurança do código e simular um fluxo de trabalho real com Pull Requests.

Cenário

Você acabou de ser contratado(a) como Desenvolvedor(a) em uma startup de tecnologia. A equipe está sofrendo porque códigos com "bugs" estão sendo enviados para o repositório principal com frequência. Sua primeira missão é criar um "Guarda-Costas" automatizado usando o GitHub Actions para testar a API toda vez que alguém tentar enviar um código novo.

Parte 1: Preparação

1. Utilize o modelo do repositório para montar o seu repo.
2. Crie repo remoto e suba o modelo (repo local > repo remoto).
3. **Ação no GitHub:** Vá imediatamente para a aba **Actions** no seu repositório. Acompanhe a bolinha amarela rodando. Clique no processo para ver o log do servidor (o Runner) até que ele fique **Verde (✅ Success)**.

Parte 2: O Dia em que a Produção Quase Caiu (Simulando um Erro)

O verdadeiro poder do CI não é quando ele dá certo, é quando ele **barra um erro**.

1. Volte ao seu código local e abra o arquivo `main.py`.
2. O desenvolvedor "estagiário" fez uma alteração acidental na nossa função de soma. Altere o retorno da função para subtrair em vez de somar:

```
@app.get("/somar/{a}/{b}")
def somar(a: int, b: int):
    # ERRO INTENCIONAL AQUI: altere de + para -
    return {"resultado": a - b}
```

3. Faça o commit e o push dessa alteração.
4. **Ação no GitHub:** Vá para a aba **Actions**. Assista ao Workflow falhar (❌).
5. **O Diagnóstico:** Clique na execução que falhou. Vá até o passo "Rodar Testes" e expanda o terminal. Responda (anote para discussão na aula):
   - **Qual foi a mensagem exata de erro que o Pytest imprimiu?**
   - **O que ele esperava receber e o que ele recebeu de fato?**

Parte 3: Trabalhando como Profissional (Pull Requests)
Na vida real, não fazemos "push" direto na `main`. Usamos ramificações (branches). Vamos ver como o CI protege o código nesse cenário.

1. Corrija o bug no `main.py` localmente (volte para `a + b`), faça o commit e o push para a `main` voltar a ficar verde.
2. No GitHub, vá em **Settings** > **Branches** (ou Rulesets) e adicione uma regra de proteção para a branch `main`:
   - Marque **Require a pull request before merging**.
   - Marque **Require status checks to pass before merging** e digite `verificar-api` (o nome do nosso Job) na barra de pesquisa para exigi-lo. Salve.
3. No seu terminal, crie uma nova branch: `git checkout -b feature-multiplicacao`.
4. No arquivo `main.py`, crie uma nova rota chamada `/multiplicar/{a}/{b}`.
5. No arquivo `test_main.py`, crie um teste para essa rota, mas **faça o teste falhar de propósito** (ex: espere que `2 * 2` seja 5).
6. Faça commit e push da nova branch (`git push origin feature-multiplicacao`).
7. Vá no GitHub e abra um **Pull Request**.
8. **Observe a Mágica:** Note que o botão verde de "Merge" estará **BLOQUEADO** pelo GitHub porque o seu CI (verificar-api) falhou. O "Guarda-Costas" funcionou!
9. Para finalizar com sucesso: corrija o teste na sua máquina, faça um novo push na mesma branch. O Pull Request vai atualizar sozinho, os testes vão rodar de novo, ficar verdes e o botão de Merge será liberado. Faça o Merge!

Parte 4: Evoluindo o Pipeline (Aplicando strategy matrix)
Você garantiu que o código funciona perfeitamente na versão 3.11 do Python. Mas a sua startup acabou de descobrir que alguns clientes vão rodar essa API em servidores mais antigos com Python 3.10, e outros nos mais novos com Python 3.12.

Como garantir que o código funciona em **todas** essas versões sem ter que copiar e colar o arquivo YAML inteiro três vezes? A resposta é a `strategy matrix`.

1. Volte para a sua branch `main` no terminal (`git checkout main` e `git pull`).
2. Abra o seu arquivo `.github/workflows/ci.yml`.

3. Adicione o bloco `strategy` e a `matrix` logo abaixo de `runs-on: ubuntu-latest`.
4. Utilize essas versões do python `python-version: ['3.10', '3.11', '3.12']`
4. Altere a linha `python-version: '3.11'` para usar a variável dinâmica `${{ matrix.python-version }}`.

Parte 5: O Teste de Integração (Rodando a API no Runner)
Nós testamos o código com o `pytest`, mas será que o servidor da API realmente "liga" sem dar erro na nuvem? Vamos adicionar um passo no workflow para subir a API em background (segundo plano) e fazer uma requisição real simulando um usuário.

1. No seu arquivo `ci.yml`, vá até o final (logo abaixo do passo do Pytest).
2. Adicione um novo passo (`step`) para rodar o FastAPI e usar o comando `curl` (um navegador de terminal) para verificar se ele responde:

```
      - name: Subir a API e testar requisição (Integração)
        run: |
          # O '&' no final roda a API em background para não travar o Runner
          fastapi run main.py --port 8000 &

          # Dá 3 segundos para o servidor ligar completamente
          sleep 3

          # O '-f' faz o CI falhar se a API retornar erro (ex: 404 ou 500)
          curl -f http://localhost:8000/
          curl -f http://localhost:8000/somar/10/20
```

3. Faça o commit e envie para o GitHub.
4. **O Resultado Final:** Abra os logs do Actions. Você verá a sua API iniciando dentro da máquina virtual do GitHub e o comando `curl` acessando as suas rotas e retornando o JSON com sucesso! Você acabou de criar um teste de caixa-preta automatizado.